

RAP Cloud 課程 8 (期末) : Fiori Elements 基礎與整合——為什麼 Draft 才有完整 CRUD 畫面

Lecture

這一課要證明的事

前七課把 RAP 的後端機制 (CUD、Determination、Validation、Action、Draft、Composition、Service Binding / Publish) 逐一驗證過，但 rc07 拿到的 Fiori Elements 畫面完全沒有 `@UI.*` annotation，List Report 只能看到框架自動生成的一整排技術欄位。這一課要做兩件事：① 幫 `ZI_RC01_TASK` 加上 `@UI.*` annotation 跟一個真正的 Value Help，讓畫面變成正常排版的 App；② 過程中意外挖到這門課最重要的收尾發現——**Fiori Elements 標準範本的完整 CRUD (Create / Edit 按鈕) 其實是綁定 Draft 機制設計的，非 Draft 實體天生就拿不到，靠實測 `ZI_RC01_TASK` (非 Draft) 跟 `ZI_RC05_NOTE` (Draft) 兩相對照證實。**

`@UI.*` Annotation : Metadata Extension 語法

查證官方文件 `ABENCDS_F1_ANNOTATE_VIEW` 確認這個環境用 `ANNOTATE ENTITY` (不是舊式 `ANNOTATE VIEW`)，Metadata Extension 建立走 Eclipse 精靈：對著 CDS View 右鍵 → New → Metadata Extension → Templates 選 `Annotate Entity`。

```
@Metadata.layer: #CUSTOMER

@UI.headerInfo: {
  typeName: 'Task',
  typeNamePlural: 'Tasks',
  title: { type: #STANDARD, value: 'description' }
}

annotate entity ZI_RC01_TASK
  with
  {
    @UI.facet: [
      { id: 'GeneralInfo', purpose: #STANDARD, type: #IDENTIFICATION_REFERENCE, label:
'General Information', position: 10 }
    ]

    @UI.selectionField: [ { position: 10 } ]
    @UI.lineItem: [ { position: 10 } ]
    @UI.identification: [ { position: 10 } ]
    task_id;

    ...

    @UI.lineItem: [
```

```

    { position: 30 },
    { type: #FOR_ACTION, dataAction: 'markDone', label: 'Mark Done' }
  ]
  @UI.identification: [
    { position: 30 },
    { type: #FOR_ACTION, dataAction: 'markDone', label: 'Mark Done' }
  ]
  @Consumption.valueHelpDefinition: [ { entity: { name: 'ZI_RC08_STATUS_VH', element:
'status' } } ]
  status;
}

```

- **@UI.headerInfo** / **@UI.facet** 寫在 **annotate entity ... with** 區塊外面 (跟 **@Metadata.layer** 同一層) / 裡面 (當作不綁欄位的浮動標記) ——這點延續舊 On-Premise 課程已經記過的規則，這裡用官方 /DMO/FSA_C_RootTP 範例 (查證 [abap-fiori-showcase](#) 文件) 再次確認一致。
- **@UI.lineItem** / **@UI.identification** 各自控制 **List Report** 表格欄跟 **Object Page** 顯示，**position** 決定排序；Action 按鈕 (**type: #FOR_ACTION**) 是額外塞進同一個陣列的一筆記錄，掛在哪個欄位上不影響按鈕出現的位置 (List Report 按鈕在工具列、Object Page 按鈕在標題列)，只是語法上要找一個地方寫。
- **@Consumption.valueHelpDefinition** 直接寫在欄位前面 (元素層級 annotation)，指向一個外部 Value Help 提供者：{ **entity**: { **name**: '<提供資料的 CDS Entity>', **element**: '<對應欄位>' } }。

Value Help : CDS Custom Entity + IF_RAP_QUERY_PROVIDER

status 只有 **O** (Open) / **D** (Done) 兩個合法值 (rc03 的 Validation 規則)，沒有現成資料庫表可以查——官方標準做法是用 **CDS Custom Entity** (**define custom entity**，不對應任何資料庫表，**metadata** 啟用後不會產生實體資料庫物件) 搭配一個實作 **IF_RAP_QUERY_PROVIDER** 的 Class，在程式碼裡動態產生資料：

```

@EndUserText.label: 'Value help for status'
@ObjectModel.query.implementedBy: 'ABAP:ZCL_RC08_STATUS_VH'
define custom entity ZI_RC08_STATUS_VH
{
  @EndUserText.label: 'Status'
  key status      : abap.char(1);
  @EndUserText.label: 'Status Text'
  status_text : abap.string;
}

```

```

METHOD if_rap_query_provider~select.
  DATA lt_result TYPE STANDARD TABLE OF zi_rc08_status_vh.
  lt_result = VALUE #( ( status = 'O' status_text = 'Open' )
    ( status = 'D' status_text = 'Done' ) ).

  io_request->get_paging( )->get_page_size( ).
  io_request->get_paging( )->get_offset( ).

```

```

io_request->get_sort_elements( ).
TRY.
    io_request->get_filter( )->get_as_ranges( ).
    CATCH cx_rap_query_filter_no_range.
ENDTRY.

IF io_request->is_total_numb_of_rec_requested( ).
    io_response->set_total_number_of_records( lines( lt_result ) ).
ENDIF.
IF io_request->is_data_requested( ).
    io_response->set_data( lt_result ).
ENDIF.
ENDMETHOD.

```

⚠ 這裡踩到一個很值得記錄的坑，分兩階段才排除乾淨：第一版只呼叫了 `is_data_requested( )` / `is_total_numb_of_rec_requested( )` 就直接 `set_data`，語法檢查 / 啟用完全正常，但透過 Eclipse 內建 Swagger UI 測 `GET /StatusVH` 直接回 **501 RAP_RUNTIME/004**，訊息「Query not fully covered by implementation: Call to method if_rap_query_request~get_paging missing」。補上呼叫 `get_paging( )` 之後重測，Swagger 直接測 OK，但 Fiori Elements 的 Value Help 對話框（透過 `$batch` 呼叫）又冒出同一種錯誤，這次是 **`get_sort_elements missing`**。

根本原因：RAP Query Provider 的執行期有一套「完整性檢查」——呼叫端在這次請求裡實際觸碰到的每一個查詢面向（分頁 / 排序 / 篩選），**Provider** 都必須明確呼叫對應的 **getter** 方法確認自己知道這件事，即使你的實作選擇忽略這個值（因為資料集太小不需要真的分頁/排序/篩選）。Swagger 直接測 `/StatusVH` 送出的查詢面向比較單純，沒有觸發 `$orderby` 要求；但 Fiori Elements Value Help 元件送出的實際請求會多帶排序參數，才會踩到 `get_sort_elements`。教訓：只用 **Swagger** 測過一輪不代表所有呼叫端都會通過，**Fiori Elements** 元件送出的實際 **OData** 請求可能比手動測試更完整，值得用瀏覽器開發者工具的 **Network** 分頁直接看 `$batch` 裡的真實請求 / 回應來排查，而不是只看畫面上「有沒有資料」這種間接跡象。

`@Metadata.allowExtensions` 是 Metadata Extension 能不能掛上去的前提

幫 `ZI_RC05_NOTE`（`rc05` 建立時完全沒想到會需要 UI annotation）加 Metadata Extension，第一次啟用直接報 Annotation 'Metadata.allowExtensions' missing in 'ZI_RC05_NOTE'——查證官方文件 `ABENCDS_F1_ANNOTATE_VIEW`：「entity 必須在原始碼裡帶 `Metadata.allowExtensions` 且預設值是 `true`」。 `ZI_RC01_TASK` 從 `rc01` 建立時就順手加了這個 annotation（當時只是照抄慣例，沒特別解釋原因），這次終於用上：回頭幫 `ZI_RC05_NOTE` 的 CDS View 本體補上 `@Metadata.allowExtensions: true`，重新啟用兩個物件（先 CDS View 再 Metadata Extension）就過了。這是一個「當初隨手加的 **annotation**，這一課才真正發揮作用」的具體案例，也提醒之後新建 **CDS View** 時，如果預期以後可能要加 **Metadata Extension**，建議一開始就加上這個 **annotation**。

🏆 核心發現：Draft 才是 Fiori Elements 標準範本「完整 CRUD 畫面」的前提

`ZRC07_SB`（曝露 `ZI_RC01_TASK`，非 Draft）Publish 之後，Preview 的 List Report 只有 **Delete** 跟自訂的 **Mark Done** Action，完全沒有 **Create**；Object Page 也沒有 **Edit**——即使 `$metadata` 證實 `POST /Task / PATCH /Task/{task_id}` 在協定層完全支援（Swagger 測過都成功）。回頭看 Service Binding 編輯器最上方，SAP 官方就直接寫明：

"oData V4 services with no draft capability will be primarily READ-ONLY"

這一課刻意選 rc05 的 `ZI_RC05_NOTE` (`with draft`;) 另外建一組 `ZRC08_SD` / `ZRC08_SB` 做對照實驗，加上跟 `ZI_RC01_TASK` 同樣模式的 `@UI.*` annotation，Publish 後 Preview：

- List Report 篩選列多出「**Editing Status**」(`All` / `Draft` / `Active`) ——這是 Draft 實體才有的標準篩選欄位，`ZI_RC01_TASK` 沒有
- List Report 工具列出現 **Create** 按鈕
- 點進 Object Page，標題列出現 **Edit** 按鈕

兩相對照，結論非常明確：SAP Fiori Elements 的 List Report / Object Page 標準範本，Create / Edit 這兩個核心 CRUD 入口是綁定 **Draft** 機制設計的——不是「非 Draft 實體的 Create/Edit 功能比較陽春」，是標準範本從設計上根本不會為非 **Draft** 實體生成這兩個按鈕，即使底層 OData 協定完全支援。這代表：「要不要幫一個 RAP BO 加上 Draft」除了 rc05 教過的「資料一致性、多步驟編輯」考量之外，還有一個很實際的附加後果——它同時決定了你能不能直接套用 **SAP** 官方標準生成的 **UI** 就有完整 **CRUD** 體驗，還是得放棄標準範本自己客製化畫面（或接受一個功能被閹割的唯讀為主 **App**）。

實務對照：Report App / 標準範本 Maintain App / 客製化 Maintain App，BDEF 分別該怎麼設計

上面的結論丟出一個很實際的問題：一般公司開發 Fiori App，有些純粹是報表（Report），有些必須能維護資料（Maintain）——同一個 BDEF + Draft 的組合，要怎麼對應這幾種不同需求？逐一查證官方文件 / 範例，三種情境各有明確答案，不是憑空推論：

情境 1：純 Report（唯讀顯示，不需要任何寫入）——查證官方 openSAP 教材的課程架構本身就是最好的證據：[week2](#)（唯讀 List Report App）整週的步驟是「建 DB Table → CDS Data Model → Projection → 加 UI Metadata → 建 Service → 加權限」，完全沒有 **BDEF**；[week3](#) 標題直接是「Enabling the **Transactional Behavior of an App**」，這才是 BDEF 第一次出場。查證官方語法文件 `ABENCDS_SERVICE_DEFINITIONS` 拿到更精確的機制說明：

Service definitions can expose **regular CDS entities** and **root entities of RAP business objects**：曝露 regular CDS entity（沒有 BDEF 的一般 CDS View）只能做查詢；曝露 RAP BO 的 root entity（有 BDEF）才能做 transactional write。

結論：純 Report App 完全不需要 BDEF，CDS View 也不需要 root 關鍵字，直接拿一個普通的 define view entity 建 Service Definition / Service Binding 就能產生一個 Fiori App。完整步驟：

1. CDS View 本體：`define view entity ZI_RCxx_REPORT as select from <表/其他View> { ... }` ——不用 `root`（`root` 是給 RAP BO Composition Root 用的標記，純查詢不需要），不用建 BDEF
2. Service Definition：`expose ZI_RCxx_REPORT as <alias>;` ——這就是官方文件講的「曝露 regular CDS entity」情境
3. Service Binding：跟這門課一路的做法一樣，選 OData V4 - UI（要 Fiori 畫面）或 OData V4 - Web API（純資料串接，不需要 UI），Publish
4. `@UI.*` Metadata Extension：一樣可以加 `headerInfo` / `facet` / `lineItem` / `selectionField` 排版 List Report，只是完全不會有任何 `{ type: #FOR_ACTION, ... }` 這種按鈕標記，因為沒有 BDEF、沒有 Action 可以掛
5. Preview：List Report 純顯示，連 **Delete** 都不會出現——這點要跟 `ZI_RC01_TASK` 的情況分清楚：`ZI_RC01_TASK` 雖然沒有 Draft、拿不到 Create/Edit，但它還是有 **BDEF**（`create;/update;/delete;`）

都宣告了) · 所以還是有 Delete 跟 Mark Done 按鈕；完全沒有 BDEF 的純 Report CDS View · 是徹底的唯讀，連 Delete 都不存在，這是比「非 Draft」更乾淨的唯讀狀態

這門課的 rc01 (`ZI_RC01_TASK` 剛建好、BDEF 還沒寫) 其實就短暫處於這個狀態——只是當時還沒有 Service Binding，沒有機會透過畫面驗證。

情境 2：要用官方標準範本自動生成完整 **Create / Edit / Delete** (跟這一課的 `ZRC08_SB` 一樣) ——查證整個 RAP BDL 語法文件 (`ABENBDL_WITH_DRAFT` / `ABENRAP_DRAFT_HANDLING_GLOSRY` 等) · **RAP** 完全沒有「**Sticky Session**」這種語法 (這是舊 SAP Gateway V2 / CAP 才有的機制，ABAP RAP 的 BDL 文法裡找不到對應關鍵字)。 `with draft;` 是 RAP 讓官方標準範本自動生成 Create / Edit 按鈕的**唯一**路徑，沒有第二條路可以繞——這就是這一課已經實測證實的結論，這裡補上「查過官方文件確認真的沒有替代方案」這一步。

情境 3：需要維護資料，但不想要 **Draft** 那套「可以存一半、多 **Session** 續編、按 **Activate** 才真正生效」的語意 (例如簡單的設定型維護畫面，填完就直接存檔生效，不需要草稿概念) ——BDEF 依然宣告 `create;/update;/delete;` (非 Draft，跟這一課的 `ZI_RC01_TASK` 一樣，OData 協定層完全支援，rc07 已用 Swagger 證實)，但不能指望框架自動生按鈕，實務上有兩條路：

- 放棄標準範本，自己刻 UI：用 Fiori Elements 的 Custom Page / manifest 擴充點手動加一個 Create 按鈕去打 `POST`，或整個改用 Freestyle SAPUI5——工程量最大，但完全不用碰 Draft 的額外機制 (Draft Table、五個標準 Action、`%is_draft`)。
- 用 RAP 的 **Factory Action** (查證官方範例 `ABENBDL_ACTION3_ABEXA`)：BDEF 宣告 `factory action <名稱> [1];` (instance-bound，複製一筆既有實例的值) 或 `static factory action <名稱> [1];` (static，不需要來源實例，直接帶預設值建一筆新的)，本質上內部一樣是走 **CREATE**，只是包成一個具名 Action 對外曝露。因為這一課已經證實 Action 按鈕 (`@UI.lineItem: [{ type: #FOR_ACTION, ... }]`) 在非 **Draft** 實體上完全正常顯示 (`ZI_RC01_TASK` 的 Mark Done 按鈕)，Factory Action 可以借同一招掛出一個「New」風格的按鈕，繞開框架綁定 Draft 才給 Create 的限制——代價是使用者體驗變成一個明確命名的 Action 按鈕 (如「New Task」)，不是標準內建的「+」圖示，但換來完全不用管理 Draft Table / Draft Action 這一整套機制。

情境	BDEF 設計	標準範本會不會自動生按鈕
純 Report	不建 BDEF，只曝露 CDS View	唯讀，不適用 Create/Edit 的問題
標準 Maintain App	<code>create;/update;/delete;</code> + <code>with draft;</code>	會，Create / Edit 全自動生成 (這一課 <code>ZRC08_SB</code> 已驗證)
客製化 Maintain App (不要 Draft)	<code>create;/update;/delete;</code> ，非 Draft，+ Factory Action 或自訂 UI	不會自動生，要自己做 (Factory Action 按鈕或客製化畫面)

學習目標

- 能寫出 `@UI.headerInfo` / `@UI.facet` / `@UI.lineItem` / `@UI.identification` / `@UI.selectionField` 的基本語法，知道 headerInfo/facet 跟 lineItem/identification 分別寫在 `annotate entity ... with` 區塊的外面/裡面
- 知道 Action 按鈕用 `{ type: #FOR_ACTION, dataAction: '<action名稱>', label: '...' }` 掛在 `@UI.lineItem` / `@UI.identification` 陣列裡，可以跟該欄位本身的其他標記共存
- 能用 `@Consumption.valueHelpDefinition` 搭配 CDS Custom Entity (`define custom entity`) + `IF_RAP_QUERY_PROVIDER` 實作一個程式碼驅動、不需要資料庫表的 Value Help

- 知道 RAP Query Provider 的「完整性檢查」機制：呼叫端實際觸碰到的每個查詢面向（分頁 / 排序 / 篩選）都要呼叫對應 getter 確認知情，即使選擇忽略回傳值，不然會報 `RAP_RUNTIME/014`；知道用 Swagger 測過一輪不代表涵蓋所有真實呼叫情境，Fiori Elements 元件的實際請求可能觸發更多面向
- 知道 `@Metadata.allowExtensions: true` 是 CDS View 能不能掛 Metadata Extension 的前提，缺了會在啟用 Metadata Extension 時直接報錯
- 能講出這門課最重要的收尾結論：OData V4 UI 服務如果背後實體沒有 Draft，Fiori Elements 標準範本的 List Report / Object Page 天生不會生成 Create / Edit 按鈕（即使協定層完全支援），這是 SAP 官方在 Service Binding 編輯器裡就寫明的行為；要有標準範本的完整 CRUD 體驗，Draft 是必要前提
- 能對照三種實務情境選擇對應的 BDEF 設計：純 Report（不建 BDEF，只曝露 CDS View）、標準 Maintain App（`create;/update;/delete;` + `with draft;`，唯一能讓框架自動生 Create/Edit 按鈕的路，RAP 沒有 Sticky Session 這種替代機制）、客製化 Maintain App（非 Draft + Factory Action 按鈕或自己刻 UI，代價是要自己處理按鈕/畫面）

物件清單

物件	名稱	型別
<code>ZI_RC01_TASK</code> 的 Metadata Extension	<code>ZI_RC01_TASK</code>	DDLX/EX
Value Help Custom Entity	<code>ZI_RC08_STATUS_VH</code>	DDLX/DF (Custom Entity)
Value Help Query Provider Class	<code>ZCL_RC08_STATUS_VH</code>	CLAS/OC
<code>ZI_RC05_NOTE</code> 的 Metadata Extension (新增)	<code>ZI_RC05_NOTE</code>	DDLX/EX
<code>ZI_RC05_NOTE</code> CDS View (補 <code>@Metadata.allowExtensions</code>)	<code>ZI_RC05_NOTE</code>	DDLX/DF
Service Definition (曝露 <code>ZI_RC05_NOTE</code>)	<code>ZRC08_SD</code>	SRVD/SRV
Service Binding (OData V4 - UI，已 Publish)	<code>ZRC08_SB</code>	SRVB/SVB

沿用 rc02-04 的 `ZI_RC01_TASK` / rc07 的 `ZRC07_SD`（追加曝露 `ZI_RC08_STATUS_VH`）、rc05 的 `ZI_RC05_NOTE`。套件：`ZRAPCLOUD`。所有物件空殼由使用者在 Eclipse ADT 建立，Claude 用 MCP 寫入內容並驗證；Publish 由使用者手動完成。

驗證方式

1. `get_abap_diagnostics` 確認所有物件無語法錯誤，`abap_activate` 全部 `Activation successful`
2. Eclipse 內建 Swagger UI：`GET /StatusVH` 回傳 `O/Open`、`D/Done` 兩筆（修完兩輪 `RAP_RUNTIME/014` 之後）
3. `ZRC07_SB` Preview：`status` 欄位篩選有 Value Help 下拉選單（Search and Select 分頁），選 `D` 能正確篩出對應資料
4. `ZRC08_SB` Preview（`ZI_RC05_NOTE`，Draft）：List Report 有 **Create** 按鈕 + **Editing Status** 篩選欄位；Object Page 有 **Edit** 按鈕——對照 `ZRC07_SB`（`ZI_RC01_TASK`，非 Draft）完全沒有這兩個按鈕，證實 Draft 是標準範本完整 CRUD 的前提

思考題

1. 講義的「實務對照」表格列出三種情境 (Report / 標準 Maintain / 客製化 Maintain) 。如果要幫 `ZI_RC01_TASK` 加一個 Factory Action (例如 `static factory action createDefault [1];` , 用預設值建一筆新 Task) , BDEF 跟 Metadata Extension 各要改哪裡? (提示: BDEF 語法可以直接照抄 `ABENBDL_ACTION3_ABEXA` 的 `static factory action` ; UI 那邊要照抄這一課 `markDone` 掛 `@UI.lineItem/@UI.identification` 的 `{ type: #FOR_ACTION, ... }` 寫法, 這一課還沒實際做過, 是留給你的動手練習)
2. `IF_RAP_QUERY_PROVIDER` 的完整性檢查這一課只踩到 `get_paging` / `get_sort_elements` 兩個。如果 Value Help 的篩選欄位 (Status Text) 被使用者實際輸入文字查詢, 你覺得會不會再冒出 `get_filter` 相關的完整性檢查要求? 要怎麼設計一個實驗驗證 (提示: 這一課的 `TRY...CATCH cx_rap_query_filter_no_range` 已經呼叫過 `get_filter()->get_as_ranges()` , 但沒有真的照篩選條件過濾 `lt_result` , 這個實驗可以順便補上這個缺口)
3. 這門課從 rc01 到 rc08 , 走過 CDS View Entity 、 `strict(2)` 、 CUD 、 Determination/Validation 、 Action 、 Draft 、 Composition 、 Service Binding 、 UI Annotation——如果要用一句話跟同事介紹「這個 Cloud 環境跟舊 On-Premise 系統的關鍵差異」, 你會怎麼講? (提示: 不是「語法比較新」這麼籠統, 具體想想 rc02 (白名單 Dump) 、 rc07 (Swagger 直接測) 、這一課 (Draft/CRUD) 分別代表哪一種「舊系統做不到、這裡做得到」的能力)

答案

見 `zi_rc01_task.ddlx.abap` (`ZI_RC01_TASK` 的完整 UI Annotation) 、 `zi_rc08_status_vh.ddls.abap` (Value Help Custom Entity) 、 `zcl_rc08_status_vh.clas.abap` (Query Provider , 含兩輪除錯後的完整 capability 呼叫) 、 `zi_rc05_note.ddls.abap` (補上 `@Metadata.allowExtensions`) 、 `zi_rc05_note.ddlx.abap` (`ZI_RC05_NOTE` 的 UI Annotation) 、 `zrc08_sd.srvd.abap` / `zrc08_sb.srvb.xml` (曝露 `ZI_RC05_NOTE` 的 Service Definition / Binding , 含 Draft vs 非 Draft 對照的完整發現記錄) 、 `zrc07_sd.srvd.abap` (更新後追加曝露 `ZI_RC08_STATUS_VH`) 。SAP 端物件套件 `ZRAPCLOUD` 。實測結果: `ZRC08_SB` Preview 確認 List Report 有 Create 、 Object Page 有 Edit , `ZRC07_SB` 對照組確認兩者皆無——RAP Cloud 課程 (rc01 ~ rc08) 全課程結案。